Studio Nx

Backend(FastAPI/Python) 82 — DevNote 모델

2026-10-12

Backend(FastAPI/Python) 82 — `DevNote` 모델 (나머지 컬럼)

항목	내용
파일	`studionx/backend/app/models/dev_note.py`
작업	P1-21 공지 API와 관리자 화면: `dev_notes` 테이블과 짝이 되는 모델 채우기 (PRD 01 FR-114, FR-115)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	(P1-21 첫 항목) · 83 `list_published()` (다음)

1. 하는 일

뼈대에는 `note_id`와 `author_id` 두 칸만 있었습니다. DB의 `dev_notes` 테이블(`DB.md` §3.1, `01_phase1_account.sql`)과 같도록 **나머지 8칸**을 채웠습니다. 이 모델을 서비스(83, 84)와 관리자 화면(86 이후)이 씁니다.

칸	타입	뜻
`title`	`String(150)`	제목
`summary`	`String(300)`	카드에 보이는 짧은 요약
`content`	`MEDIUMTEXT`	본문 (Markdown)
`is_published`	`Boolean`, 기본 0	공개 여부. 0은 초안이라 메인 페이지에 보이지 않습니다
`published_at`	`DATETIME(3)`, 비어 있을 수 있음	공개 시각. 메인 페이지 정렬 기준이며, 미래로 두면 그때까지 숨는 예약 공개입니다
`created_at`	`DATETIME(3)`, 기본 `CURRENT_TIMESTAMP(3)`	만든 시각
`updated_at`	`DATETIME(3)`, 기본 현재, 바뀔 때 갱신	고친 시각
`deleted_at`	`DATETIME(3)`, 비어 있을 수 있음	소프트 삭제 (관리자 화면에서 지워도 행은 남습니다)

2. 언제 쓰나

호출하는 곳	용도
83 `list_published()`, 84 `get_published()`	공개된 공지를 읽을 때
`DevNoteAdmin` (관리자 화면)	공지를 작성, 수정, 공개할 때

3. 코드

import

```
from sqlalchemy import BigInteger, Boolean, FetchedValue, ForeignKey, String, text
from sqlalchemy.dialects.mysql import DATETIME, MEDIUMTEXT
```

나머지 컬럼

```
title: Mapped[str] = mapped_column(String(150))
summary: Mapped[str] = mapped_column(String(300))
content: Mapped[str] = mapped_column(MEDIUMTEXT)
is_published: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
published_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
created_at: Mapped[datetime] = mapped_column(
    DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
)
updated_at: Mapped[datetime] = mapped_column(
    DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
)
deleted_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
```

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

문법 풀이

식	뜻
`Mapped[str]`	이 칸은 파이썬에서 글자(`str`)입니다. `NULL`이 안 되는 칸입니다
`Mapped[datetime | None]`	날짜이거나 비어 있을 수 있는 칸입니다 (`NULL` 허용)
`String(150)`	최대 150글자의 `VARCHAR(150)`입니다
`MEDIUMTEXT`	최대 약 16MB의 긴 글자 칸입니다. 일반 `Text`(`TEXT`)는 약 64KB라서 긴 공지 본문에는 모자랍니다
`Boolean`	파이썬에서는 `True`, `False`이고 MySQL에서는 `TINYINT(1)`입니다
`server_default=text("0")`	값을 안 넣으면 **DB가** 0을 넣습니다
`DATETIME(fsp=3)`	밀리초(소수점 3자리)까지 저장합니다
`server_onupdate=FetchedValue()`	행이 바뀌면 DB가 `updated_at`을 갱신하므로, 파이썬은 바뀐 값을 다시 읽어 오라는 표시입니다

4. 흐름

새 공지를 저장할 때 `title`, `summary`, `content`, `author_id`만 넣으면 `is_published`는 0(초안), `created_at`과 `updated_at`은 DB가 현재 시각으로 채웁니다. 공개하려면 `is_published`를 켜고 `published_at`을 정합니다.

5. 왜 이렇게

선택	이유
`is_published`를 `TINYINT(1)`이 아니라 `Boolean`으로	뼈대 주석의 지시이기도 하고, `TINYINT(1)`로 쓰면 값이 `0`, `1`로 나와서 `is False` 비교가 맞지 않습니다 (시험에서 확인했습니다). MySQL에는 같은 `TINYINT(1)`로 만들어집니다
`content`를 `MEDIUMTEXT`로	DB.md가 정한 대로이며, 공지는 Markdown 본문이 길어질 수 있습니다
시각 기본값을 DB가 채움	다른 모델(`Inquiry`, `ProjectSession`)과 같습니다. 연결 시간대를 UTC로 고정했으므로(P1-17) 저장되는 값은 UTC입니다
`deleted_at` 소프트 삭제	`DB.md`가 `dev_notes`를 소프트 삭제 대상으로 정했습니다
관계(`relationship`)는 만들지 않음	지금은 `author_id` 번호만 필요합니다. 작성자 이름을 보여 줄 일이 생기면 그때 더합니다

6. 시험 (`tests/test_dev_note_model.py`, 4개)

시험	결과
10칸이 실제 DB와 같음 (이름, 순서, `NULL` 허용, FK 1개)	통과입니다
최소 칸만 넣고 저장하면 초안(`False`), 공개 시각과 삭제 시각이 비어 있음, `created_at`은 시간대 없는 값	통과입니다
약 20만 글자의 한글과 기호 본문, 150자 제목과 300자 요약, 밀리초 공개 시각이 그대로 저장됨	통과입니다
없는 작성자 번호는 DB가 거절(외래키)	통과입니다

4개 모두 통과했습니다. 백엔드 전체는 **572개 통과**, `ruff`도 통과했습니다.